Architectural Research Report: Agentic Orchestration Harness (AOH) for Visual Studio Code

Deliverable A: Executive Findings

The Agentic Orchestration Harness (AOH) is an enterprise-grade orchestration framework designed to execute across Visual Studio Code custom agent runtimes[1]. Designed for multi-tiered organizational environments—such as the Global Markets Technology (GMT) division with its Alpha and Beta teams—AOH establishes operational governance, state persistence, subagent delegation control, and telemetry capture[3].

Designing a resilient orchestration harness requires navigating a fundamental architectural boundary within Visual Studio Code: the operational split between the **Local Extension Host Harness** and the **Agent Host Harness** (which executes out-of-process runtimes for GitHub Copilot, Anthropic Claude, and OpenAI Codex)[2]. The Local harness runs directly inside the VS Code Extension Host process, offering native lifecycle hooks (.github/hooks/*.json), agent-scoped hook commands, and synchronous tool interception[1]. In contrast, Agent Host sessions run on dedicated, isolated background processes[5]. VS Code settings controlling local hooks, such as chat.useHooks and chat.hookFilesLocations, do not configure or govern Agent Host sessions[5].

Because Visual Studio Code custom agents do not natively provide persistent long-term memory APIs or cross-session state databases[6], AOH introduces an explicit workspace memory architecture[4]. Transient session context is augmented by file-backed state stores, structured capability registries, and Model Context Protocol (MCP) state adapters[4]. Delegation topology is governed using explicit allow-lists (agents frontmatter property combined with the agent tool)[1] and structured handoffs (handoffs frontmatter property)[1]. Non-invocable specialist agents (user-invocable: false) are shielded from direct user interaction while remaining fully accessible to authorized team orchestrators[1].

Deliverable B: Official Feature and Limitation Matrix

The matrix below documents every customization mechanism supported by Visual Studio Code, detailing supported paths, scope resolution, configuration schemas, runtime harness compatibility, and security boundaries[1].

Customization Mechanism	Default Supported Location	Scope & Resolution Rules	Discovery & Activation	Key Schema / Frontmatter Fields	Harness Compatibility & Status	Limitations & Security Consider

	s					ations
Custom Agents [cite: 1]	Workspace: .github/agents/*.agent.md or .github/agents/*.md User: User profile storage Org: GitHub central settings[1].	Workspace, User, or Organization scope. Monorepo root resolution via chat.useCustomizationsInParentRepositories[1].	Auto-detected from .github/agents/. Selectable via Chat UI dropdown or programmatically invoked as subagents[1].	name, description, tools, agents, model, user-invocable, disable-model-invocation, handoffs, hooks[1].	Supported in Local and Agent Host (Copilot, Claude, Codex)[1]. hooks frontmatter key is Preview in Local harness[1].	File placement does not enforce security isolation; model compliance determines execution unless intercepted by hooks[4].
Agent Skills [cite: 8]	Workspace: .github/skills/<name>/SKILL.md, .claude/skills/, .agents/skills/ User: ~/.copilot/skills/[8].	Workspace, User, or Extension contribution (chatSkills)[8].	Auto-loaded based on prompt relevance or slash command /skills. Directory name must match name[8].	name, description, argument-hint, user-invocable, disable-model-invocation, context (inline or fork)[8].	Open standard supported across Local, Copilot, Claude, and Codex runtimes[8].	Manual namespace prefixes cause silent failure[8]. context: fork isolates execution within a subagent[8].
Custom Instructions [cite: 9]	Workspace: .github/copilot-instructions.	Workspace, User (~/.copilot/instructions), or	Automatically included in prompts	name, description, applyTo (glob	AGENTS.md is cross-compatible[9]	Large instruction files consume model

	md, .github/instructions/*.instructions.md, AGENTS.md[9].	Organization scope[9].	or dynamically matched via applyTo glob patterns[9].	pattern)[9]. AGENTS.md requires no frontmatter[9].	.instructions.md supported in Local and Copilot[9].	context window[4]. Conflicting rules resolve by narrowest scope precedence[9].
Prompt Files [cite: 10]	Workspace: .github/prompts/*.prompt.md User: VS Code profile storage[10].	Workspace or User profile scope[10].	Invoked via / slash command or Command Palette[10].	agent, model, tools, description, ${input:var} placeholders[10].	**Deprecated** for Agent Host sessions[10]. Functional in Local harness only[10].	Teams must migrate prompt files to Agent Skills for future Agent Host compatibility[10].
Lifecycle Hooks [cite: 5]	Workspace: .github/hooks/*.json or .github/hooks/hooks.json Agent: Frontmatter hooks key[1].	Workspace, Plugin, or Agent-scoped[1].	Loaded via chat.hookFilesLocations or active agent scope[5].	Events: SessionStart, UserPromptSubmit, PreToolUse, PostToolUse, PreCompact, SubagentStart, SubagentStop, Stop[5].	**Preview** status in Local harness[5]. Local hook settings do not configure Agent Host sessions[5].	Hooks run with full Extension Host process permissions; shell scripts must be audited for injection risks[5].

Tool Sets [cite: 11, 12]	Defined via *.jsonc files configured in Chat settings[11].	Workspace or User scope[11].	Referenced via #toolset-name in prompts or listed in agent tools array[11].	tools (array of tool names), description, icon[11].	Supported in Local and Copilot runtimes[1][1].	Groups tools logically; does not alter underlying tool permissions[11].
MCP Servers [cite: 4, 7]	.vscode/mcp.json or enterprise-managed configurations[7].	Workspace, User, or Enterprise Organization scope[7].	Auto-started on workspace load or on-demand tool invocation[7].	Standard MCP JSON format defining stdio/sse transports and tool definitions[7].	Universal support across Local, Copilot, Claude, and Codex[2].	Stdio server execution requires workspace trust and corporate policy approval[7].
Persistent Memory [cite: 6]	Custom AOH paths: .aoh/memory/ or external MCP database[4].	Managed by AOH Framework conventions[4].	Explicitly loaded via Agent Skills, instructions, or MCP tool calls[4].	Custom AOH Schema: JSON document containing entity, key, context, and TTL fields.	Architectural pattern; native API is absent in VS Code[6].	Stale memory can pollute prompt context; secrets and unmasked PII must be excluded.

Deliverable C: Recommended Repository Skeleton and File Structure

The repository structure below differentiates natively discovered VS Code paths from custom directories managed exclusively by the AOH framework[1].

```
gmt-aoh-workspace/
├── .github/
│   ├── agents/ # [Native VS Code] Custom agent definitions (.agent.md)
│   │   ├── gmt-orchestrator.agent.md # Division Orchestrator
│   │   ├── alpha-orchestrator.agent.md # Alpha Team Orchestrator
│   │   ├── alpha-sec-auditor.agent.md # Alpha Specialist Agent
│   │   ├── beta-orchestrator.agent.md # Beta Team Orchestrator
│   │   └── beta-perf-analyst.agent.md # Beta Specialist Agent
│   ├── skills/ # [Native VS Code] Workspace agent skills
│   │   ├── gmt-compliance-check/ # Division-wide Skill
│   │   │   ├── SKILL.md # Skill metadata & instructions
│   │   │   └── resources/ # Regulatory policy checklists
│   │   ├── alpha-k8s-deploy/ # Alpha Team Skill
│   │   │   ├── SKILL.md
│   │   │   └── scripts/ # Deployment helper scripts
│   │   └── beta-db-migration/ # Beta Team Skill
│   │   ├── SKILL.md
│   │   └── templates/ # SQL migration templates
│   ├── instructions/ # [Native VS Code] Scoped instructions
│   │   ├── gmt-core.instructions.md # Division rules (applyTo: "")
│   │   ├── alpha-service.instructions.md # Alpha rules (applyTo: "services/alpha/")
│   │   └── beta-service.instructions.md # Beta rules (applyTo: "services/beta/")
│   ├── hooks/ # [Native VS Code] Local harness lifecycle hooks
│   │   └── aoh-hooks.json # Central hook dispatch configuration
│   ├── copilot-instructions.md # [Native VS Code] Global Copilot instructions
│   └── AGENTS.md # [Native VS Code] Cross-agent fallback instructions
├── .aoh/ # [AOH Framework Managed] Custom framework directory
│   ├── config/ # Organization & team settings
│   │   ├── division.json # GMT division policy and boundaries
│   │   ├── team-alpha.json # Alpha team configuration
│   │   └── team-beta.json # Beta team configuration
│   ├── registries/ # Capability and agent metadata manifests
│   │   ├── agent-registry.json # Manifest of all division/team agents
│   │   └── skill-registry.json # Manifest of registered skills and dependencies
│   ├── memory/ # Persistent Agent Memory Banks
│   │   ├── division/ # Global architecture decisions & shared knowledge
│   │   ├── alpha/ # Alpha team memory bank
│   │   └── beta/ # Beta team memory bank
│   ├── state/ # Workflow execution tracking
│   │   └── active-runs.json # Running workflow stages and execution status
│   ├── audit/ # Telemetry and audit logs
│   │   └── trace.jsonl # Append-only JSONL execution stream
│   └── schemas/ # JSON validation schemas
```

```
│   ├── hook-payload.schema.json # Schema for hook payloads
│   └── memory-bank.schema.json # Schema for state persistence
├── scripts/ # [AOH Framework Managed] Executable tooling & scripts
│   ├── aoh/ # Framework control scripts
│   │   ├── hook-handler.py # Primary hook dispatcher (stdin -> audit log)
│   │   ├── validate-manifests.py # Schema validation utility
│   │   └── memory-sync.py # Syncs memory state to workspace files
│   ├── alpha/ # Alpha team specific scripts
│   │   └── validate-k8s-spec.sh
│   └── beta/ # Beta team specific scripts
│   └── check-schema-drift.sh
└── .vscode/
├── settings.json # Workspace VS Code settings
└── mcp.json # MCP Server configurations for state/memory
```

Explanation of Directories and Ownership

1. **.github/agents/ (VS Code Discovered)**: Discovered natively by VS Code[1]. Every .agent.md file in this directory is exposed in the Chat UI agent selector or registered as a callable subagent[1]. Subfolders within .github/agents/ are not natively parsed for agent discovery in standard VS Code configurations[1].
2. **.github/skills/ (VS Code Discovered)**: Discovered natively by VS Code[8]. Each subdirectory must contain a SKILL.md file whose name frontmatter property matches the directory name exactly[8].
3. **.github/instructions/ (VS Code Discovered)**: Discovered natively by VS Code[9]. Contains *.instructions.md files equipped with applyTo glob patterns to automatically inject instructions based on target file paths[9].
4. **.github/hooks/ (VS Code Discovered)**: Discovered natively by the VS Code Local harness[5]. Stores *.json configuration files that map lifecycle events to script executables[5].
5. **.aoh/ (AOH Framework Managed)**: Managed entirely by AOH runtime scripts and MCP adapters[4]. VS Code treats this directory as standard workspace files. It houses configuration manifests, audit logs, workflow finite state machines, and agent memory banks[4].
6. **scripts/ (AOH Framework Managed)**: Houses Python and Bash execution tools invoked by lifecycle hooks, agent skills, or MCP servers[5].

Script Compatibility and Cross-Platform Portability

To guarantee seamless execution across Windows PowerShell, macOS, and Linux Git Bash environments, script invocations must adhere to strict platform standards[5]:

- **Path Resolvers**: All relative paths within agent definitions, skill files, and hook maps must use Unix-style forward slashes (/)[8]. VS Code internally converts forward slashes to system-native separators across all supported operating systems[5].
- **Platform Neutrality**: Hook executables should invoke cross-platform runtimes (such as python3 or node) rather than OS-dependent shell scripts[5]. Where native platform shell

scripts are required, platform-specific overrides must be specified using the standard windows, linux, and osx hook keys[5].

- **Relative Path Resolution**: Paths declared in custom agents resolve relative to the agent file location[1]. Paths in hooks.json resolve relative to the workspace root[5].

Deliverable D: Frontmatter and Configuration Specifications

Below are complete, production-ready specification files for the GMT orchestrator hierarchy, specialist agents, and team skills[1].

GMT Division Orchestrator

File Location: .github/agents/gmt-orchestrator.agent.md
[cite: 1]
name: GMT Orchestrator
description: Global Markets Technology division orchestrator. Coordinates cross-team workflows between Alpha and Beta teams.
model:

- gpt-5
- claude-3-5-sonnet
  tools:
- agent
- search/codebase
- search/changes
- read/problems
  agents:
- alpha-orchestrator
- beta-orchestrator
  user-invocable: true
  disable-model-invocation: false
  handoffs:
- label: Initiate Alpha Deployment Review
  agent: alpha-orchestrator
  prompt: Review Kubernetes deployment specifications for Alpha team services.
  send: false
- label: Initiate Beta Database Audit
  agent: beta-orchestrator
  prompt: Audit database migration scripts and check for schema drift.
  send: false

You are the Division Orchestrator for Global Markets Technology (GMT). Your mandate is to coordinate cross-team initiatives, enforce divisional governance, and route work items to authorized team orchestrators.

Operational Rules:

1. You do NOT directly edit source code or execute destructive database commands.

2. Delegate specialized domain tasks to alpha-orchestrator or beta-orchestrator using the #tool:agent tool.
3. Before initiating any cross-team handoff, verify that the active workspace state in #file:.aoh/state/active-runs.json contains no blocking compliance flags.
4. Always summarize the execution outputs returned by subagents before completing the session.

Alpha Team Orchestrator

File Location: .github/agents/alpha-orchestrator.agent.md
[cite: 1]
name: Alpha Orchestrator
description: Team Alpha Lead Orchestrator. Manages Alpha service architectures, deployment pipelines, and security audits.
model: claude-3-5-sonnet
tools:

- agent
- search/codebase
- edit
- terminal/runCommand
  agents:
- alpha-sec-auditor
  user-invocable: true
  disable-model-invocation: false
  handoffs:
- label: Trigger Security Audit
  agent: alpha-sec-auditor
  prompt: Perform a static security audit on modified Alpha services.
  send: true

You are the Lead Orchestrator for Team Alpha. You are responsible for engineering execution within services/alpha/.

Delegation Policy:

- For security assessments, code analysis, and vulnerability scans, delegate exclusively to alpha-sec-auditor.
- Ensure all deployment workflows execute the alpha-k8s-deploy skill procedures.
- Maintain team state logs in .aoh/memory/alpha/state.json.

Alpha Team Specialist Agent

File Location: .github/agents/alpha-sec-auditor.agent.md
[cite: 1]
name: Alpha Security Auditor
description: Specialist agent for static security auditing and vulnerability scanning in Team Alpha services.
tools:

- search/codebase

- read/problems
  agents: []
  user-invocable: false
  disable-model-invocation: false
  hooks:
  PostToolUse:
  - type: command
    command: "python3 scripts/aoh/hook-handler.py --event PostToolUse --agent alpha-sec-auditor"

You are a specialized read-only security auditing agent for Team Alpha.

Core Constraints:

1. You have read-only access to codebase search and diagnostic tools. You cannot edit files or run raw shell commands.
2. Report all detected vulnerabilities categorized by OWASP Top 10 classifications.
3. Structure your final output strictly as a JSON security report block.

GMT Division-Wide Skill

File Location: .github/skills/gmt-compliance-check/SKILL.md
[cite: 8]

name: gmt-compliance-check description: Evaluates codebase changes against Global Markets Technology regulatory compliance and data hygiene standards. Use when reviewing code prior to pull request submission. argument-hint: "[target-directory]" user-invocable: true disable-model-invocation: false context: inline

GMT Compliance Check Procedure

Follow these sequential steps to verify divisional compliance:

1. **Secret Scanning**: Search the target directory for hardcoded API keys, JWT tokens, private RSA keys, or database credentials.
2. **Data Privacy Inspection**: Ensure that no Personally Identifiable Information (PII) or financial account numbers are emitted to standard log files without masking.
3. **Licensing Verification**: Confirm all third-party dependencies imported in package files comply with the divisional open-source allow-list (Apache-2.0, MIT, BSD-3-Clause).
4. **Audit Logging**: Write compliance scan results to .aoh/audit/trace.jsonl using the standard audit event schema.

Team-Owned Skill

File Location: .github/skills/alpha-k8s-deploy/SKILL.md
[cite: 8]

name: alpha-k8s-deploy description: Packages, validates, and dry-runs Kubernetes deployment manifests for Team Alpha microservices. argument-hint: "[service-name] [environment]" user-invocable: true disable-model-invocation: false context: fork

Alpha Kubernetes Deployment Procedure

This skill executes within an isolated subagent context (context: fork).

Execution Steps:

1. Validate the target Helm chart values against .github/skills/alpha-k8s-deploy/resources/schema.json.
2. Execute a dry-run helm template rendering using the terminal tool: helm template ./services/alpha/charts --values ./services/alpha/values-dev.yaml
3. Check the output for unparsed template variables or missing namespace definitions.
4. Return a structured status summary (SUCCESS or FAILED) to the calling agent.

Deliverable E: Lifecycle Hooks and Execution Observability Architecture

Lifecycle Events Matrix

The table below catalogs every lifecycle hook supported by the VS Code Local Extension Host runtime, detailing available payload fields, control mechanisms, and reliability guarantees[5].

Event Name	Supported Harness	Trigger Context	Payload Data Exposed via stdin[cite: 5]	Control Capability	Reliability & Guarantees	Primary AOH Purpose
SessionStart	Local Harness[5]	First prompt is	sessionId, timestam	Observe & inject	Deterministic. Fires	Initialize run state

[cite: 5]		submitted in a session[5].	p, workspaceFolders[5].	initial context string via stdout[5].	exactly once per session initialization[5].	in .aoh/state/active-runs.json and start trace stream[5].
UserPromptSubmit [cite: 5]	Local Harness[5]	User sends a chat message[5].	sessionId, promptText, user, timestamp[5].	Observe, modify prompt context, or block request[5].	Synchronous. Blocks agent execution until hook exits[5].	Audit prompt text, enforce prompt policy, redact sensitive inputs.
PreToolUse [cite: 5]	Local Harness[5]	Agent selects a tool, prior to invocation[5].	sessionId, agentId, toolName, toolArgs[5].	Allow, Deny (block execution), or modify tool inputs[5].	Synchronous enforcement gate[5].	Deterministic guardrail: block unsafe shell commands or file edits[5].
PostToolUse [cite: 5]	Local Harness[5]	Tool completes execution successfully[5].	sessionId, agentId, toolName, toolArgs, toolResult[5].	Observe, validate result, or inject follow-up context[5].	Fires only if tool execution completes without throwing unhandled exception[5].	Capture modified file paths, record tool execution status in trace log.
PreCompact [cite: 5]	Local Harness[5]	Model context window	sessionId, currentContextTok	Save critical context	Best effort before	Persist transient memory

		compaction triggered[5].	ens[5].	before history summarization[5].	memory compaction takes place[5].	state to .aoh/memory/ before history loss[5].
SubagentStart [cite: 5]	Local Harness[5]	Parent agent invokes a subagent[5].	sessionId, parentAgentId, subagentId, initialPrompt[5].	Observe & track hierarchy relationships[5].	Fires synchronously upon subagent activation[5].	Record parent-child execution correlation in audit log.
SubagentStop [cite: 5]	Local Harness[5]	Subagent completes execution[5].	sessionId, subagentId, status, summaryOutput[5].	Observe or require additional iteration[5].	Fires prior to transferring control back to parent agent[5].	Update stage status in workflow state model.
Stop [cite: 5]	Local Harness[5]	Main agent execution finishes or terminates[5].	sessionId, agentId, stopReason (completed, cancelled)[5].	Observe session completion[5].	Guaranteed to fire upon task exit unless process dies violently[5].	Finalize trace logging and update active runs manifest.

Telemetry Record Schema and Audited Sample

AOH normalizes all captured hook events into an append-only JSON Lines trace stream located at .aoh/audit/trace.jsonl. Below is the official JSON schema and a sample event record captured during tool invocation.

```
JSON
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "AOHAuditEvent",
 "type": "object",
 "required": [
  "event_id",
  "timestamp",
  "session_id",
  "event_type",
  "harness",
  "agent_context"
 ],
 "properties": {
  "event_id": { "type": "string", "format": "uuid" },
  "timestamp": { "type": "string", "format": "date-time" },
  "session_id": { "type": "string" },
  "workflow_id": { "type": "string" },
  "stage_id": { "type": "string" },
  "event_type": {
   "type": "string",
   "enum": [
    "SessionStart",
    "UserPromptSubmit",
    "PreToolUse",
    "PostToolUse",
    "PreCompact",
    "SubagentStart",
    "SubagentStop",
    "Stop"
   ]
  },
  "harness": { "type": "string", "enum": ["Local", "AgentHost-Copilot", "AgentHost-Claude", "AgentHost-Codex"] }
```

```
  "agent_context"
    "type" "object"
    "required" "current_agent"
    "properties"
      "current_agent" "type" "string"
      "parent_agent" "type" "string" "null"
      "division" "type" "string"
      "team" "type" "string"
    
  
  "payload"
    "type" "object"
    "properties"
      "tool_name" "type" "string"
      "tool_args" "type" "object"
      "tool_result_summary" "type" "string"
      "exit_code" "type" "integer"
      "redacted_fields" "type" "array" "items" "type" "string"
    
  
  "metrics"
    "type" "object"
    "properties"
      "execution_duration_ms" "type" "number"
      "estimated_input_tokens" "type" "integer"
      "estimated_output_tokens" "type" "integer"
    
  
  
```

JSON

```
 "event_id" "c4b8a1d2-8f3e-4d9a-9b1c-7e5f3a1b9c2d"
 "timestamp" "2026-10-07T14:22:10.452Z"
 "session_id" "sess-8829104-gmt"
 "workflow_id" "wf-alpha-deploy-902"
 "stage_id" "stage-sec-audit"
```

```
  "event_type": "PostToolUse",
  "harness": "Local",
  "agent_context": {
    "current_agent": "alpha-sec-auditor",
    "parent_agent": "alpha-orchestrator",
    "division": "GMT",
    "team": "Alpha"
  },
  "payload": {
    "tool_name": "search/codebase",
    "tool_args": { "query": "password_hash", "path": "services/alpha/" },
    "tool_result_summary": "Found 0 occurrences of insecure hashing routines.",
    "exit_code": 0,
    "redacted_fields": []
  },
  "metrics": {
    "execution_duration_ms": 142.0,
    "estimated_input_tokens": 420,
    "estimated_output_tokens": 85
  }
}
```


Reality of Token and Prompt Capture

The VS Code Local hook API passes tool names, parameters, and invocation outputs directly to the script via standard input (stdin)[5]. However, the exact assembled prompt string sent to the underlying model provider (which incorporates system wrappers and hidden context) as well as exact API token consumption counts are not fully exposed in hook payloads[5].

To establish token and prompt accounting without relying on brittle transcript scraping, AOH employs a multi-tiered approach:

1. **Local Harness**: Computes estimated token counts within hook-handler.py using standard tokenizer libraries applied to user prompts and tool payload text strings.
2. **Agent Host Harness**: Utilizes provider-level telemetry endpoints or local API proxy wrappers to capture exact token usage and billing data[5].

Workspace Hook Configuration File

File Location: .github/hooks/aoh-hooks.json

[cite: 5]

```JSON
{
```

```
{
  "$schema": "https://code.visualstudio.com/schemas/hooks.json",
  "description": "AOH Workspace Lifecycle Hooks Configuration",
  "hooks": {
    "SessionStart": [
      {
        "type": "command",
        "command": "python3 scripts/aoh/hook-handler.py --event SessionStart",
        "timeout": 5000
      }
    ],
    "UserPromptSubmit": [
      {
        "type": "command",
        "command": "python3 scripts/aoh/hook-handler.py --event UserPromptSubmit",
        "timeout": 5000
      }
    ],
    "PreToolUse": [
      {
        "type": "command",
        "command": "python3 scripts/aoh/hook-handler.py --event PreToolUse",
        "timeout": 10000
      }
    ],
    "PostToolUse": [
      {
        "type": "command",
        "command": "python3 scripts/aoh/hook-handler.py --event PostToolUse",
        "timeout": 10000
      }
    ],
    "Stop": [
      {
        "type": "command",
        "command": "python3 scripts/aoh/hook-handler.py --event Stop",
        "timeout": 5000
      }
    ]
  }
}
```


Deliverable F: Orchestration, Delegation, and Memory

Architecture

Hierarchy and Delegation Dynamics

Orchestrating work across division and team boundaries requires strict execution limits to prevent circular calls or unauthorized cross-team actions[1]. AOH enforces these boundaries using explicit frontmatter directives rather than filesystem directory containment[1]:

1. **Subagent Delegation vs. Handoffs**:
   - **Subagent Delegation**: Managed programmatically using the agent tool[1]. The calling agent retains session ownership, invokes the target agent, receives the child agent's execution summary, and proceeds with its task[1]. Managed via the frontmatter agents list[1].
   - **Handoffs**: Rendered as interactive UI buttons upon task completion[1]. Handoffs transition primary user focus from one agent to another while transferring context[1].
2. **Preventing Circular and Unbounded Delegation**:
   - Setting agents: [] explicitly prohibits an agent from delegating to any subagent[1]. Specialist agents MUST specify agents: [][1].
   - Team orchestrators may only invoke subagents explicitly listed in their agents allow-list[1].
   - Self-referential delegation requires setting chat.subagents.allowInvocationsFromSubagents to true[1]. AOH leaves this setting disabled by default to eliminate recursive loops[1].

Capability Registry Manifest

File Location: .aoh/registries/agent-registry.json

```
JSON
{
 "division": "GMT",
 "version": "1.0.0",
 "orchestrators": {
  "division": "GMT Orchestrator",
  "teams": {
   "Alpha": "Alpha Orchestrator",
   "Beta": "Beta Orchestrator"
  }
 },
 "agents": [
  {
   "id": "gmt-orchestrator",
   "file": ".github/agents/gmt-orchestrator.agent.md"
```

```
    "role": "Division Orchestrator",
    "allowed_subagents": ["alpha-orchestrator", "beta-orchestrator"],
    "visibility": "public"
  },
  {
    "id": "alpha-sec-auditor",
    "file": ".github/agents/alpha-sec-auditor.agent.md",
    "role": "Team Alpha Specialist",
    "allowed_subagents": [],
    "visibility": "subagent-only"
  }
]
```

Persistent Memory Architecture

Because VS Code custom agents lack native persistent memory across sessions[6], AOH establishes a three-tier state architecture[4]:

1. **Transient Session Context**: In-memory conversation history and active window context[4]. Cleared upon chat reset[13].
2. **Workspace Memory Banks (File-Backed)**: JSON documents stored in .aoh/memory/{division|team}/. Stored facts, architecture decisions, and operational context are loaded into prompts using Markdown links or #file: references[1].
3. **Dynamic Memory MCP Server**: Configured in .vscode/mcp.json[7]. Exposes state tools (aoh_read_memory, aoh_write_memory) for semantic search and state updates across workspaces[4].

Workflow Execution FSM State Manifest

File Location: .aoh/state/active-runs.json

```
JSON
{
  "active_run_id": "run-20261007-001",
  "division": "GMT",
  "target_team": "Alpha",
  "workflow_name": "alpha-service-deployment",
  "current_stage": "STAGE_2_SECURITY_AUDIT",
  "status": "IN_PROGRESS",
  "stages": [
```

```
    "stage_id": "STAGE_1_BUILD",
    "agent": "alpha-orchestrator",
    "status": "COMPLETED",
    "exit_code": 0
  },
  {
    "stage_id": "STAGE_2_SECURITY_AUDIT",
    "agent": "alpha-sec-auditor",
    "status": "IN_PROGRESS",
    "exit_code": null
  }
],
"human_approval_required": false
}
```


Deliverable G: Security, Operational Risks, and Practical Workarounds

1. **Least Privilege Tool Allocation**:
   - Planning and auditing agents must be granted read-only tools (e.g., search/codebase, read/problems) to prevent unapproved edits[1].
   - Broad command tools (such as terminal/runCommand) should be reserved exclusively for team orchestrator agents[1].
2. **Terminal Execution Controls**:
   - Built-in VS Code terminal approvals require user confirmation prior to running shell commands[8]. AOH configurations must operate within these safety controls.
3. **Data Masking and Credential Protection**:
   - Telemetry hooks (hook-handler.py) automatically redact high-entropy strings, API keys, JWTs, and private keys before committing trace logs to .aoh/audit/trace.jsonl.
4. **Workspace Trust and Local Hooks Security**:
   - Executable hooks run with the full security permissions of the VS Code process[5]. Local hooks (chat.useHooks) execute only within explicitly trusted workspaces[5].

Practical Architectural Workarounds

- **Harness Parity Workaround**: Because Agent Host sessions do not execute local .github/hooks/*.json commands[5], critical governance rules are duplicated within custom Agent Skills (SKILL.md) and MCP servers[4]. Skills execute across all runtimes uniformly[8].
- **Cross-Session Memory Workaround**: To maintain state without native memory support[6], AOH uses an MCP state server defined in .vscode/mcp.json[7]. Agents update and query state via dedicated MCP tools[4].
- **Deterministic Approval Gates**: Rather than relying solely on model prompt compliance, a PreToolUse hook intercepts sensitive shell commands (e.g., database updates) and

halts execution until user confirmation is logged[5].

Deliverable H: Implementation Blueprint and Task Roadmap

Minimum Framework Skeleton Components

A minimal functional deployment of the AOH framework requires the following core files:

1. .github/agents/gmt-orchestrator.agent.md
   [cite: 1]
2. .github/agents/alpha-orchestrator.agent.md
   [cite: 1]
3. .github/skills/gmt-compliance-check/SKILL.md
   [cite: 8]
4. .github/hooks/aoh-hooks.json
   [cite: 5]
5. .aoh/registries/agent-registry.json
6. .aoh/audit/trace.jsonl
7. scripts/aoh/hook-handler.py

Sequential Ten-Task Implementation Roadmap

Task Order	Title	Objective & Action Items
Task 1	Workspace Skeleton Setup	Create the required directory structure (.github/agents/, .github/skills/, .github/hooks/, .aoh/, scripts/)[1].
Task 2	VS Code Environment Configuration	Configure .vscode/settings.json to enable local hooks (chat.useHooks: true) and configure workspace trust settings[5].
Task 3	Schema Definition	Author JSON validation schemas in .aoh/schemas/ for hook payloads and memory banks.

Task 4	Telemetry Handler Development	Implement scripts/aoh/hook-handler.py to parse stdin payloads and append trace entries to .aoh/audit/trace.jsonl[5].
Task 5	Lifecycle Hook Registration	Create .github/hooks/aoh-hooks.json mapping SessionStart, PreToolUse, and PostToolUse to the handler script[5].
Task 6	Division Orchestrator Deployment	Author .github/agents/gmt-orchestrator.agent.md and establish the initial capability registry manifest[1].
Task 7	Team Orchestrators Deployment	Author .github/agents/alpha-orchestrator.agent.md and specialist definitions with restricted subagent lists (agents: [])[1].
Task 8	Core Skills Creation	Implement division and team skills (SKILL.md) utilizing both inline and fork execution contexts[8].
Task 9	MCP Memory Adapter Setup	Configure .vscode/mcp.json to launch the workspace state adapter for persistent memory management[4].
Task 10	System Validation Execution	Run the validation suite to confirm agent discovery, hook execution, tool interception, and state persistence.

Deliverable I: Empirical Validation Plan and Open Questions

Validation Experiment Matrix

Experiment ID	Title	Setup Condition	Execution Action	Expected Observation	Success Criteria
EXP-01	Agent Discovery[1]	.github/agents/gmt-orchestrator.agent.md present[1].	Open Chat UI and inspect Agent dropdown menu[1].	"GMT Orchestrator" appears in the available custom agents list[1].	Agent loads successfully and renders input description placeholder[1].
EXP-02	Subagent Delegation[1]	gmt-orchestrator has agents: ['alpha-orchestrator'][1].	Prompt GMT agent to invoke beta-orchestrator.	Agent declines direct call or routes via approved handoff[1].	No unapproved subagent invocations occur[1].
EXP-03	Hook Execution[5]	chat.useHooks: true, .github/hooks/aoh-hooks.json active[5].	Submit a chat prompt in the Local harness[5].	UserPromptSubmit executes hook-handler.py[5].	Audit entry appended to .aoh/audit/trace.jsonl[5].
EXP-04	Tool Interception[5]	PreToolUse configured to deny commands containing rm -rf[5].	Prompt agent to run a destructive terminal command.	Hook returns denial payload; command execution is blocked[5].	Command is halted prior to terminal execution[5].
EXP-05	State	MCP state	Store a fact;	Agent	Fact

	Persistence [4]	server declared in .vscode/mcp.json[7].	clear chat session; query for stored fact.	retrieves fact using MCP state tools[4].	correctly retrieved across session boundaries[4].
EXP-06	Forked Skill Context[8]	Skill configured with context: fork[8].	Invoke skill using / slash command[8].	Skill executes within an isolated subagent context[8].	Main conversation context remains unpolluted[8].
EXP-07	Cross-Platform Paths[5]	Script paths defined using forward slashes (/)[8].	Test setup across Windows PowerShell and Linux.	Path references resolve identically on both platforms[5].	Zero file resolution errors across OS environments[5].

Open Questions and Verification Targets

1. **Agent Host Lifecycle Alignment**: Monitor future VS Code releases to determine when Agent Host runtimes will support native lifecycle hooks identical to the Local Extension Host[5].
2. **Subagent Stack Depth**: Verify runtime behavior and stack boundaries when nested subagent invocations exceed three levels[1].
3. **Native Token Telemetry**: Track language model API updates to evaluate when exact input/output token metrics will be exposed directly within lifecycle hook payloads[5].

Works cited

1. Custom agents in VS Code, https://code.visualstudio.com/docs/agent-customization/custom-agents
2. A Unified Experience for all Coding Agents - Visual Studio Code, https://code.visualstudio.com/blogs/2025/11/03/unified-agent-experience
3. Introduction to custom agents - Visual Studio Code, https://code.visualstudio.com/learn/customizations/4-custom-agent
4. Understand agent customization - Visual Studio Code, https://code.visualstudio.com/docs/agents/concepts/customization
5. Configure agent hooks in Visual Studio Code (Preview), https://code.visualstudio.com/docs/agent-customization/hooks
6. https://en.wikipedia.org/wiki/Memory
7. Add and manage MCP servers in VS Code, https://code.visualstudio.com/docs/agent-customization/mcp-servers
8. Use Agent Skills in VS Code, https://code.visualstudio.com/docs/agent-customization/agent-skills
9. Use custom instructions in VS Code, https://code.visualstudio.com/docs/agent-customization/custom-instructions
10. Use prompt files in VS Code, https://code.visualstudio.com/docs/agent-customization/prompt-files
11. https://code.visualstudio.com/docs/agent-customization/tool-sets
12. Use tools with agents - Visual Studio Code, https://code.visualstudio.com/docs/agents/run/tools
13. AI language models in VS Code, https://code.visualstudio.com/docs/agent-customization/language-models